Clientsideprog - JavaScript programmering

Indhold

Faglige mål 2

Målpind som dagens øvelse dækker 2

Baggrund 3

JavaScript i webapplikationer 3

Rendering (DOM) 3

Hvorfor bruge JavaScript? 3

Tips til best practice, sikkerhed og test 3

Øvelse (Ugens case) 4

1. Vælg et serverside framework 4
2. Case – Implementer HTML/CSS/JavaScript klient til en server app. 4

2.1. Implementer server app 4

2.2. Implementer klient app 5

2.3. Implementer performance sikkerhed 6

2.4. Reflektér over din løsning 6

Hvordan du bliver bedømmet 6

Faglige mål

Målpind som dagens øvelse dækker

10. **Eleven kan anvende grundlæggende JavaScript kommandoer og strukturer.**
11. **Eleven kan anvende funktioner og parameteroverførsel.**
12. **Eleven kan arbejde med JavaScript-objekter.**
13. **Eleven kan redegøre for væsentlige forskelle mellem JavaScript og et OOP sprog.**
14. **Eleven kan anvende JavaScript til at give funktionalitet til en webpage.**
15. **Eleven kan benytte jQuery og forskellige selectors til navigation og manipulation af websidens DOM.**
16. **Eleven kan benytte jQuery og AJAX til at kommunikere med en backend web Service.**
17. **Eleven kan beskrive best practise muligheder ved anvendelsen af JavaScript i forhold til både sikkerhed og performance.**
18. **Eleven har en generel viden om forskellige sikkerhedsproblemer, der kan opstå under afvikling af clientside webapplikationer, som f.eks. cross-site scripting, og kan anvende denne i sit programdesign.**
19. **Eleven kan gennem test dokumentere funktionaliteten i en udviklet webapplikation.**

Baggrund

JavaScript i webapplikationer

JavaScript programmering vil til en hvis grad overlappe web applikation programmering mod en framework (både serverside og clientside). *Derved også overlapning af teorier, f.ex:* ***render tree, DOM, komponenter*** *vs.* ***partial view*** *vs.* ***controls***.

JavaScript er et programmeringssprog, der afvikles i browseren og bruges til at gøre en webside interaktiv og dynamisk.

- Kan anvendes **uden frameworks** (ren .html), hvor JavaScript arbejder sammen med HTML og CSS.
- JavaScript kan også bruges sammen med et **framework** og en backend-server.
    - *En del af at lære at bruge JavaScript er, at lære hvor man ikke bør bruge JavaScript.*

Rendering (DOM)

- I en ren HTML application, er rendering delvis med JavaScript.
- Ren HTML eller HTML i en clientside app kan funger som en del af en serverside configuration, hvor JavaScript på klienten kan sende og modtage data fra serveren uden nødvendigvis at genindlæse hele websiden. Det kan fx ske med **Fetch/AJAX/HTMX**, hvor HTML/clientside app klienten kræver at gå igennem en **webservice/API**.

Hvorfor bruge JavaScript?

Du bruger JavaScript til eksekvering af funktioner som understøtter din applikations funktionaliteter, design din GUI, kommuniker med server via service/API med udgangspunkt i en klient app (.html og clientside frameworks).

Brug JavaScript til at flytte noget af programmets logik ud til klienten og gøre brugeroplevelsen mere interaktiv. JavaScript kan:

- anvende grundlæggende kommandoer, variabler, betingelser og løkker
- oprette **funktioner** og sende parametre til dem
- arbejde med data
- manipulere sidens **DOM** – fx ændre tekst (**rendering**), HTML, CSS og reagere på brugerinput
- håndtere events som klik, formularer og tastaturinput (**GUI**)
- validere brugerens input, før det sendes til serveren (**Validering**)

Tips til best practice, sikkerhed og test

Når JavaScript bruges i en webapplikation, skal man også tænke på både **performance, sikkerhed og test**.

Performance

- Undgå unødvendige DOM-opdateringer og netværkskald (**delvis rendering,** *render globalt vs per component med Blazor*)
- Genbrug funktioner og kode (byg controls, eventuelt partial views)
- Brug moderne, **asynkrone** API-kald, når det er relevant (Frameworks tilbyder indbygget **async metoder**)

Sikkerhed

- Stol aldrig på data fra klienten – serveren skal altid **validere** input
- Vær opmærksom på **Cross-Site Scripting (XSS) og SQL-injection (hvis input går til db)**
    - Undgå at indsætte brugerinput direkte som HTML
- Beskyt API'er og backend mod ugyldige eller manipulerede requests (**foretningsregler**)
    - Klient-side validering forbedrer brugeroplevelsen (**GUI**), men erstatter ikke server-side validering (**forretningsregler**)

Øvelse (Ugens case)

1. Vælg et serverside framework

Du skal selv vælge blandt følgende frameworks:

- En selvstændig Web App **og** en selvstændig Web Api.
- En Web App med inbygget Api.
  - *Alle .NET serverside web applikationer tilbyder Api integration.*
- Vælge hvilket Api løsning blandt følgende:
  - Minimal Api
  - Controller based Api

2. Case – Implementer HTML/CSS/JavaScript klient til en server app.

2.1. Implementer server app

Du skal **implementer en server app** med et framework så som MVC, RazorPages, Blazor server osv. (lav en ny server app, eller genbrug din tidliger serverside app fra GUI)

Din server app **skal implementer endpoints** til at modtag/send tekstbaseret-filer upload/download fra din klient.

Din server app **skal implementer GUI**, der viser en oversigt over de filer den har modtaget fra din klient app.

- Du **skal** bruge **LINQ** som løsning til læsning af din upload/download folder for indhold.
- Du skal kunne slette filen i GUIen.

2.2. Implementer klient app

1. Du skal **implementer en fil upload/download klient app** uden brug af et framework som skal **bestå udelukkende af HTML, CSS og JavaScript**.
2. Klienten skal implementer GUI så brugeren kan uploade og download tekstbaseret-filer (.txt, .pdf, docx, .csv osv) til en server (server == serverside web app).
3. Klienten GUI skal vise en list af alle filer som finds på serverens updoad/download folder så brugeren har oversight over filer man kan download.
4. Klienten skal kommunikere med din server med en teknik der understøtter delvis rendering, ved hjælp af f.ex. **jQuery AJAX** til:
    1. Send en fil til server.
    2. At hente listen over filer som den kan download.
    3. Download en fil fra server.
    4. *Opdatere GUI'en efter en ændring.*

    Eksempel:

    ```
    $.ajax({
      url: "/api/upload",
      method: "POST",
      success: function (data) {
        // Gem fil i serverens fil struktur
      },
      error: function () {
        // Log eller vis fejl
      }
    });
    ```

    Men det du opnår med Ajax() javascript teknik kan du opnå med **fetch()** javascript Teknik. Undersøg deres forskel og vælg hvilket af de 2 du vil implementer:
    - https://www.geeksforgeeks.org/javascript/difference-between-ajax-and-fetch-api/
5. Klienten skal anvende **jQuery events** til at håndtere brugerens interaktion med GUI'en.
6. Klients upload/download skal være asyncron.

2.3. Implementer performance sikkerhed

Gennemgå din Klient for performance best practis

- Undgå unødvendige DOM-opdateringer og netværkskald (**delvis rendering,** *render globalt vs per component med Blazor*)
- Genbrug funktioner og kode (byg controls, eventuelt partial views)
- Brug moderne, **asynkrone** API-kald, når det er relevant (Frameworks tilbyder indbygget **async metoder**)

Gennemgå din Klient for sikkerhed best practis

- Stol aldrig på data fra klienten – serveren skal altid **validere** input.
- Vær opmærksom på **Cross-Site Scripting (XSS) og SQL-injection (hvis input går til db)**
    - Undgå at indsætte brugerinput direkte som HTML
- Beskyt API'er og backend mod ugyldige eller manipulerede requests (**foretningsregler**)
    - Klient-side validering forbedrer brugeroplevelsen (**GUI**), men erstatter ikke server-side validering (**forretningsregler**)

2.4. Reflektér over din løsning

Du skal til bedømmelse kunne:

- Redegøre for væsentlige forskelle mellem JavaScript og et OOP sprog.
- Vis hvor og hvordan du i din kode, har implementeret løsningen til fil up-/download mod din server på klienten.
- Vis hvor din endpoints er implementeret på serveren samt forklare hvordan endpoints virker på din server og hvorfor din app både er en GUI-app samtidig med at være en Web Api.
- Forklar forskel på Controller based og Minimal Api.
- Vis et eksempel fra din app, på hvordan din apps bruger render tree, til at opdater din DOM.
- Vis hvor din jQuery implementationerne er, og hvordan de fungere for hver af implementation krav i øvelsen (stikprøve)
- Vis i koden hvad du har gjort for håndtering af performance som dækkes i afsnt "Baggrund".
- Vis i koden hvad du har gjort for håndtering af sikkerhed som dækkes i afsnt "Baggrund".

Vurdering anvendes som en del af det samlede vudering ved karaktergivning.

Hvordan du bliver bedømmet

- Du skal fremvis kørsel af din application der opfylder de stillede krav.
- Opgave indeholder en "refleksion" øvelse.
    - Du bliver bedømmet via spørgsmåler stillet med udgangspunkt i refleksionen af din øvelser.